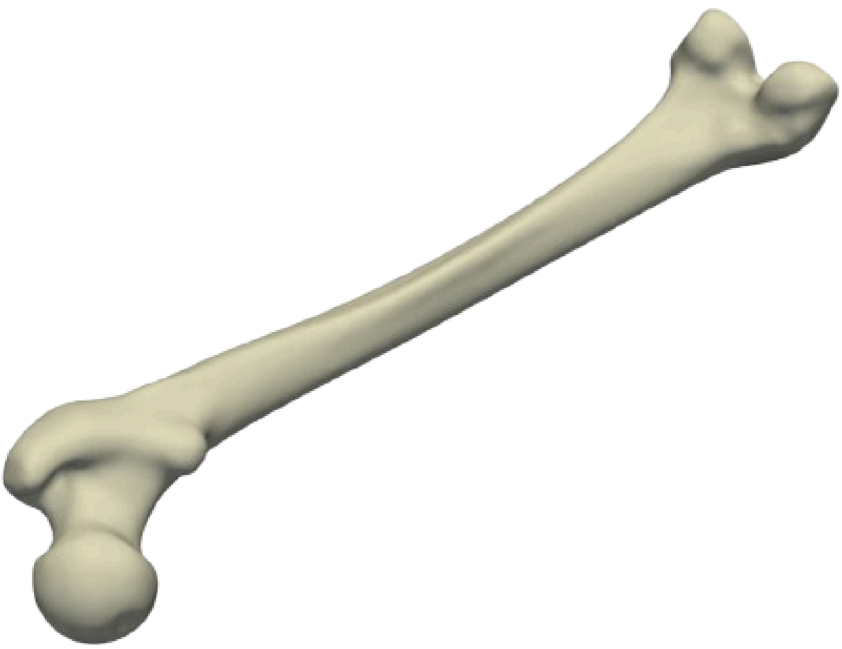
Statistical, Neural and Geometric approaches in Computational Anatomy



Grenoble INP : ENSIMAG
Master's in Applied Mathematics
Modeling Project in C++ - Marek Bucki
Spring 2026

Martin Lainé, Basile Mouret, Timothé Boyer, Malik Hacini
02.02.2026 -

Abstract

For this project, we developed a comprehensive statistical shape analysis system for femur bones. Our approach relies on two complementary methodological pillars: Principal Component Analysis (PCA), enabling linear dimensionality reduction and identification of the main variation modes, and artificial neural networks, providing the capability to capture non-linear relationships in the data. Each femur in our dataset is represented as a 3D mesh comprising 18,291 vertices, resulting in a vector of dimension 54,873 in the shape space. The main objective of this work is to build a statistical shape model capable of reducing the dimensionality of the data using an autoencoder built with neural networks, generating new anatomically plausible femur shapes. However, during our research, we were unable to guarantee biological consistency with regard to the anatomical consistency of the femurs, which is possible thanks to variation. This led us to a third avenue of research concerning the LDDMM framework.

Introduction

Here is the report on our work on “Statistical and Neural Approaches to 3D Femur Modeling.” In the field of medical imaging, understanding the variations in the shapes of different parts of the human body is a crucial issue. During our work, we focused on human femurs and asked ourselves the following question: “How can we model and compress the complex 3D shape of a femur and explore the possible variations ?”. To answer this question, we worked with a dataset of 24 femur scans represented as a dense point cloud. Our goal was to reduce this high-dimensional data into a compact set of parameters that preserve the anatomical structure. We explored three main approaches to achieve this goal: Principal Component Analysis (PCA), Neural Networks, and Large Deformation Diffeomorphic Metric Mapping (LDDMM). Each of these methods offers unique advantages and challenges in capturing the intricate variations in femur shapes. In this report, we present our methodology, results, and insights gained from applying these techniques to femur shape modeling.

1 Motivation

Modeling the 3D shape of the human femur is a central challenge in medical image analysis. The femur exhibits both global similarities and subtle anatomical variations across individuals, which are crucial for applications such as surgical planning, implant design, and pathology detection. However, each femur in our dataset is represented by a dense 3D mesh with over 18,000 vertices, resulting in extremely high-dimensional data (54,873 dimensions per shape).

The main challenge is to find a compact, informative representation of this complex shape space that preserves anatomical variability while enabling efficient analysis and generation of new, plausible femur shapes. Traditional linear methods like PCA can capture the main modes of variation, but may miss non-linear relationships that are important for biological realism. Neural networks, on the other hand, offer the potential to learn these non-linearities, but require careful design and validation to ensure anatomical plausibility.

In this project, we focus on the problem of dimensionality reduction for femur shapes: how can we compress such high-dimensional anatomical data into a small set of parameters, without losing essential information? Addressing this question is key to advancing statistical shape modeling and enabling new applications in personalized medicine.

2 PCA

2.1 Linear Principal Component Analysis (PCA)

Our first approach to understand the shape variability of femur bones is through Linear Principal Component Analysis (PCA).

2.1.1 Principle

It's clear that in general, femur bones have a similar structure, but they can vary in size, curvature, and other shape characteristics. PCA is a statistical technique that helps us to identify and quantify these variations. It's a change of basis that aims to find the directions (principal components) in which the data varies the most. By projecting the data onto these principal components, we can reduce the dimensionality of the dataset while retaining most of the variance.

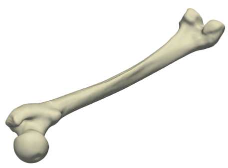


Figure 1 - 3D visualisation of a femur. We recognize a general shape

2.1.2 Change of Basis

To perform PCA, we start with a dataset of femur bone shapes represented as high-dimensional vectors. We note our N femurs $S_i \in \mathbb{R}^{3P}$ the shape vector of the i^{th} femur, where P is the number of points used to represent the shape. The first step is to compute the mean shape vector $\bar{S}$:

$$S = \frac{1}{N} \sum_{i=1}^{N} S_i \tag{1}$$

We then center the data by subtracting the mean shape from each femur shape vector and compute the covariance matrix C of the centered data:

$$C = \frac{1}{N-1} \sum_{i=1}^{N} \left(S_i - \bar{S}\right)\left(S_i - \bar{S}\right)^T \tag{2}$$

Our goal is to find a change of basis for our centered vector that all our data are uncorrelated. That means we want to find a basis where the covariance matrix is diagonal. This is achieved by finding the eigenvalues and eigenvectors of the covariance matrix C. The eigenvectors represent the directions of maximum variance (principal components), and the corresponding eigenvalues indicate the amount of variance captured by each principal component.

2.1.3 Dimensionality Reduction

Once we have the principal components, we can project the original femur shape vectors onto a lower-dimensional subspace spanned by the top K principal components. This is done by selecting the K eigenvectors $v_k$corresponding to the largest eigenvalues λ_k.

Any femur instance S_i in the dataset can be approximated as the mean shape plus a weighted sum of the principal components.

$$S_i \approx \bar{S} + \sum_{k=1}^{K} w_k v_k \tag{3}$$

where w_k correspond to the standard deviations along each principal component direction.

2.1.4 Results, visualisation and limitations

Performing PCA on our femur dataset, allows us to reduce the dimensionality from 54873 (3*18291) to 10 while still capturing a significant amount of the variance in the data.

However, PCA has its limitations. It assumes that the data lies on a linear subspace, which may not always be the case for complex shapes like femur bones. Additionally, PCA is sensitive to outliers and may not capture non-linear relationships in the data. To address these limitations, we also explored non-linear dimensionality reduction techniques, such as Neural Networks.

3 Autoencoder

3.1 Neural Networks

Neural Networks are a class of machine learning models inspired by the structure and function of biological neural networks. They are particularly well-suited for modeling complex, non-linear relationships in data. In this section, we describe the architecture and training process of the neural network implemented for our shape analysis task.

3.1.1 Modelisation of Neurons

A single artificial neuron receives an input x of size n, $x \in \mathbb{R}^n$. Each component associated with a weight. The neuron computes a weighted sum of these inputs and adds a bias term. Mathematically, this can be expressed as:

$$f(x) = b + \sum_{k=1}^{n} w_k x_k \quad x = (x_1 \; \dots \; x_n) \tag{4}$$

However, this result is just a linear combination of the inputs. To introduce non-linearity into the model, an activation function is applied to this weighted sum.

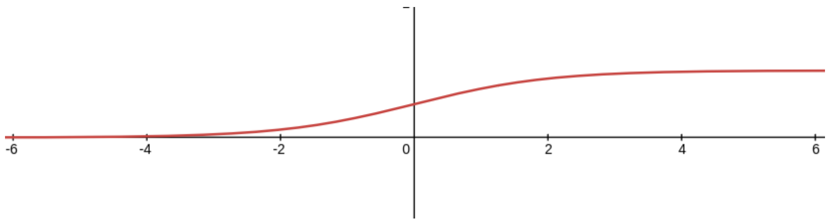


Figure 2 - Sigmoid activation function $\Phi(x) = \frac{1}{1+e^{-x}}$

Remark: The sigmoid function is one of many activation functions used in neural networks. Others include ReLU (Rectified Linear Unit), Tanh, or Softmax, each with its own advantages and applications.

Finally, the output of the neuron is given by the activation function applied to the weighted sum:

$$(\Phi \circ f)(x) = \Phi(f(x)) \tag{5}$$

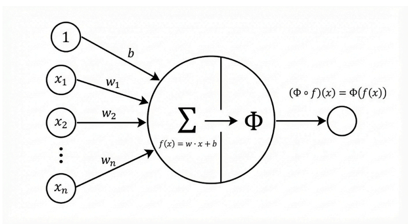


Figure 3 - Structure of an Artificial Neuron

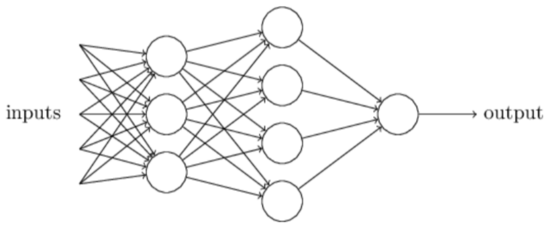


Figure 4 - Structure of a Neural Network

3.1.2 Layers and Network Architecture

Artificial neurons are organized into layers to form a neural network. A typical feedforward neural network consists of an input layer, one or more hidden layers, and an output layer. Each layer contains multiple neurons, and the output of one layer serves as the input to the next layer. However, the weights and biases of the neurons are initially set to random values and need to be optimized through a training process.

3.1.3 Backpropagation and Training

Training a neural network involves adjusting the weights and biases to minimize the difference between the predicted outputs and the actual target values. This is typically done using a method called backpropagation combined with an optimization algorithm like gradient descent.

The training process consists of the following steps:

1. **`Forward Pass`**: The input data is passed through the network layer by layer to compute the output predictions.
2. **`Loss Calculation`**: The loss function quantifies the difference between the predicted outputs and the actual target values.
3. **`Backward Pass`**: The gradients of the loss function with respect to each weight and bias are computed using the chain rule of calculus. This process is known as backpropagation.
4. **`Weight Update`**: The weights and biases are updated using the computed gradients and a learning rate, which determines the step size for each update.

3.1.4 Neural Network Implementation

For this project, we implemented a Neural Network from scratch in **`C++`**. This gave us a deep understanding of the method and the underlying computations. We started by creating a **`Vector`** and **`Matrix2D`** class for which we implemented all the necessary linear algebra methods for a neural network. We used the **`Eigen`** library to store the data as we were only interested in the mathematical implementation. We then created a **`Neural Network`** class defining fully connected neural networks with specific activation and loss functions. This class also has the methods to do a forward pass of the neural network and train it using backpropagation.

3.1.5 Autoencoder Structure

In order to reduce the dimensionality and to discover underlying relationships in the data we used an Autoencoder structure. This is a type of fully connected neural network which tries to regenerate the input data while passing through a very small layer. This layer called the latent space acts as a bottleneck for information transfer between the encoder (the part before the latent space) and the decoder (the part after it). Our input and output layers consist of 54873 neurons as we had 18291 three dimensional points for each femur. By empirical testing, we chose a latent space of size 10 as it seemed enough to capture the main component of the dataset. As we used our own implementation of neural networks, which isn't as efficient a state of the art libraries, we had to aggressively compress each layer, going from the full 54873 down to 256, 32 and finally 10 neurons for the latent space. Furthermore, in order to get faster training, we did some preprocessing by subtracting the mean femur before feeding them to the autoencoder.

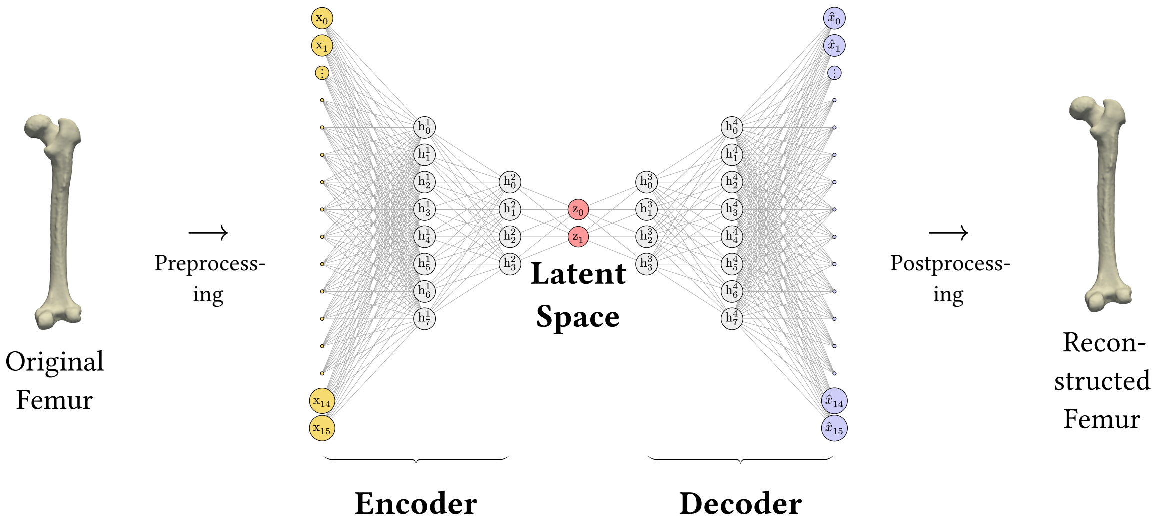


Figure 5 - Autoencoder pipeline for femur mesh reconstruction

We used the Mean Squared error for the loss function as all the point were corresponding to one another. We also tried different activation functions, and got the best results with LeakyReLU and tanh.

As we implemented the neural network and linear algebra ourselves, the training took a long time. We then focused on increasing performance.

3.2 Multi-threading

Since training the neural network is quite slow, we used multi-threading to speed up the process. We focused on parallelizing the matrix-vector multiplication (MatVec) operations, as these are computationally intensive and benefit the most from parallel execution.

3.2.1 With `std::threads`

Our first approach was to use **`std::threads`** from the C++ standard library.

3.2.1.1 Naive method

At first, we naively tried to create a thread for each operation that could be parallelized. However, the overhead from creating so many threads was far greater than any time saved. For example, for the largest matrix in our network ($54873 \times 512 = 28,094,976$ parameters), we would have created 28,094,976 threads, which is obviously not feasible. With this approach, our neural network was actually slower than the single-threaded version.

3.2.1.2 Improved approach

We then decided to split the computation among a fixed number of threads (depending on the number of CPU cores, typically 8 or 16). Each thread computes a portion of the result vector: the first thread computes the first $\frac{\text{result.size()}}{\text{num\_threads}}$ elements, the second thread the next $\frac{\text{result.size()}}{\text{num\_threads}}$ elements, and so on.

This approach worked much better, and we observed a significant speedup in the training time of our neural network. However, there was still some overhead due to thread creation and destruction at each MatVec operation.

3.2.1.3 Thread pool

As noted in Section 3.2.1.2, we were still creating and destroying too many threads. The best solution would be to implement a thread pool, where a fixed number of threads are created at the start of the program and reused for multiple MatVec operations. Unfortunately, due to time constraints, we did not implement this method, which could have further improved the performance of our neural network training.

3.2.2 OpenMP

A second approach was to use OpenMP, a popular API for parallel programming (see [1]). It provides a simple and efficient way to parallelize code, as it is close to sequential code and automatically manages thread creation and workload distribution.

3.2.3 Performance Comparison

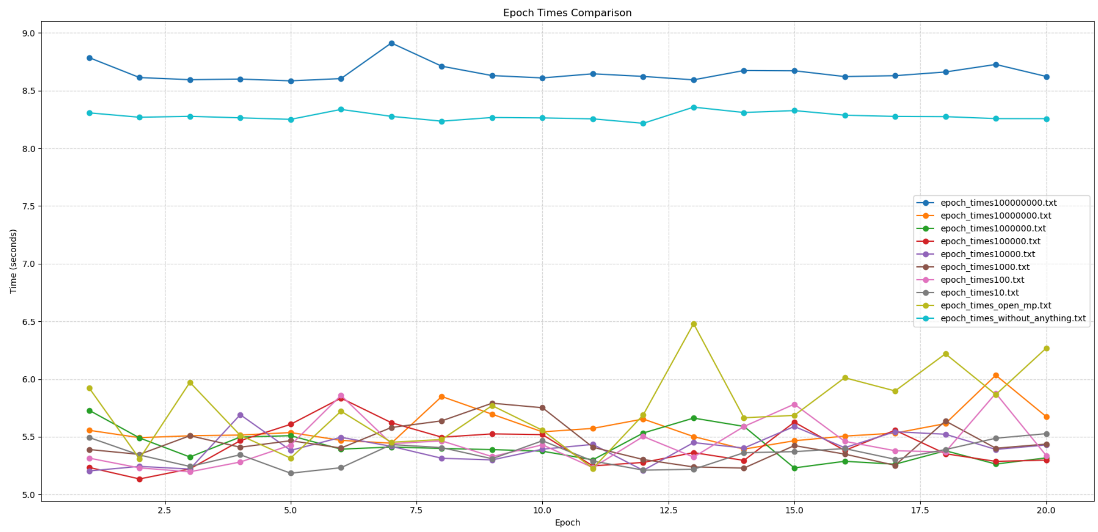


Figure 6 - Performance comparison between single-threaded and multi-threaded implementations. **Note:** The number at the end of epoch_times corresponds to the threshold parameter, which is the number of parameters in the weight matrix above which we use multithreading.

As shown in Figure 6, using multi-threading significantly reduces the time required to train the neural network: we gain about 3 seconds per epoch, which is quite significant since we train our neural network for hundreds of epochs.
Even though we achieved better results with this multi-threaded approach, as mentioned in Section 3.2.1.3, we could have obtained even greater improvements by implementing a thread pool with **`std::threads`**.

Overall, the multi-threaded implementation shows a clear advantage over the single-threaded version, especially as the size of the dataset increases. This demonstrates the effectiveness of parallelizing computationally intensive operations in our neural network training process.

Remark: We observed that the time taken by our neural network to train with OpenMP multi-threading or with our **`std::threads`** implementation is quite similar. The OpenMP method likely uses an algorithm similar to the one we implemented with **`std::threads`**.

Remark 2: As expected, with the threshold parameter set to 100,000,000, all computations are single-threaded (since the largest matrix in the neural network contains 28,094,976 parameters), so the training time matches that of the single-threaded implementation.

3.3 Optimizing Memory Allocations

The main bottleneck of our Neural Network implementation is memory bandwidth as most of the training and inference time is lost waiting for data to be loaded from memory. This meant we had to optimize the memory allocations of our computations.

In order to increase our performance, we adopted a data oriented design programming approach by increasing memory cache efficiency. As matrices are the largest data structures of our programs, we had to understand how they were stored. There are two main convention, in **`C`** multidimensional arrays are row-major, so they are stored from left to right then up to down. On the contrary, in **`Fortran`** arrays are stored in columns-major order.

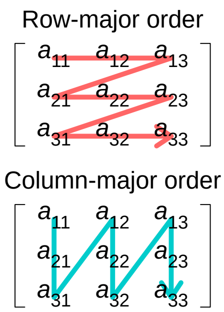


Figure 7 - Illustration of row- and column-major order by CMG Lee.

For compatibility reasons with linear algebra libraries in **`Fortran`** like **`BLAS`** and **`LAPACK`**, **`Eigen`** uses the column-major order by default. This means elements from a same columns are near one another in memory. So when we access elements of a matrix, we should iterate over elements from the same columns as they will already be loaded in the cache, resulting in faster load times.

```
for (size_t j=0; j < m_rows; ++j){
  for (size_t i=0; i < m_cols; ++i){
    // operation
  }
}
```

$\longrightarrow$

```
for (size_t j=0; j < m_cols; ++j){
  for (size_t i=0; i < m_rows; ++i){
    // operation
  }
}
```

Listing 1 - Exchanging iteration order results in 2x faster model training by reducing cache misses

We then reduced the amount of memory that was created and destroyed during the neural network methods by preallocating matrices and vectors. These are placeholders in memory created before a function call and that are reused, thus removing the overhead of allocating and destroying memory. The allocation and destruction overhead.

Finally we combined some linear algebra operations to remove temporary matrices. For example when computing $y = A^t \cdot x$ during the backpropagation, we were creating a temporary transpose matrix :

```
Matrix2D<T> W_transpose = m_weights[layer + 1].transpose();
Vector<T> weightedDelta = W_transpose * deltas[lastLayer - layer - 1];
```

In order to eliminate this, we created a new method on the **`Matrix2D`** class that combines both operations efficiently :

```
Vector<T> multiplyTranspose(const Vector<T>& vec) const {
    Vector<T> result(m_cols);

    for (size_t j = 0; j < m_cols; ++j) {
        T dot = 0;
        for (size_t i = 0; i < m_rows; ++i) {
            dot += m_data(i, j) * vec(i);
        }
        result(j) = dot;
    }
    return result;
}
```

In total, optimizing our memory usage made our training epochs eight times faster.

3.4 Tools to debug and assess the quality of the reconstruction

To help us debug and assess the quality of our neural network, we developed several Python scripts using the PyVista library [2]:

3.4.1 `visuFemur.py`

This was the first script we implemented. It allows us to visualize a femur mesh from a .OBJ file. We initially used it to understand our data, and later to debug and visualize femur meshes generated by the neural network.

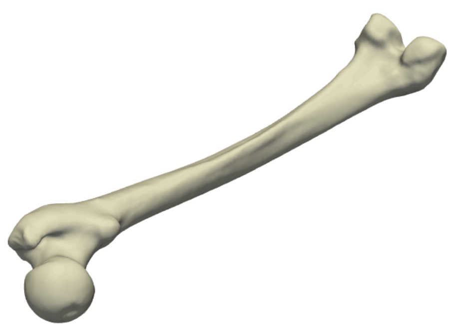


Figure 8 - Visualization of a femur mesh using **`visuFemur.py`**

3.4.2 `compare_femur.py`

We also implemented a script to compare two femur meshes, displaying a heat map of the differences (using the $\|...\|_1$ norm). We mainly used it to compare original and reconstructed femurs, to debug our neural network and assess its reconstruction quality.

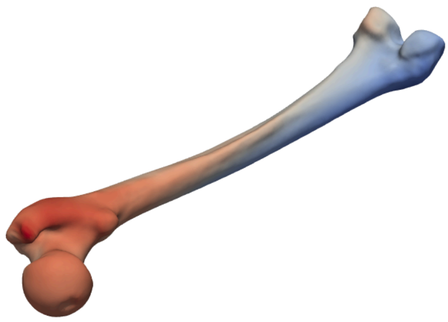


Figure 9 - Heat map of differences between two femur meshes using **`compare_femur.py`**

3.4.3 `latent_explorer.py`

Finally, we developed a script to explore the latent space of the neural network. It allows us to modify the 10 latent variables using sliders and see the corresponding reconstructed femur mesh in real time.

This tool is very useful for debugging and testing the neural network, especially for understanding how the latent space is structured and how the different latent variables affect the reconstructed femur shape. It can also be used to generate new femur shapes by exploring the latent space.

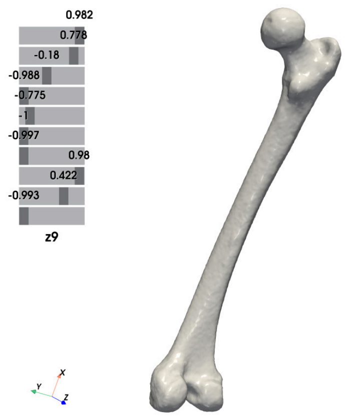


Figure 10 - Latent space explorer using **`latent_explorer.py`**

With these sliders, we can generate a wide variety of femur shapes by moving the 10 latent variables. This was very useful for testing the generative capabilities of our neural network and for checking if it could generate realistic femur shapes.

However, by moving the sliders, we also noticed that some generated femurs were not realistic. That’s why we decided to perform a PCA on the latent space, to see if we could identify directions that lead to realistic femurs (see Section 3.5).

3.5 Encoder results

The goal of the encoder is to reduce the dimensionality of the femur shapes from 54,873 to 10.

We decide to plot the data in the latent space using only these three components. To see if there is some structure in the data.

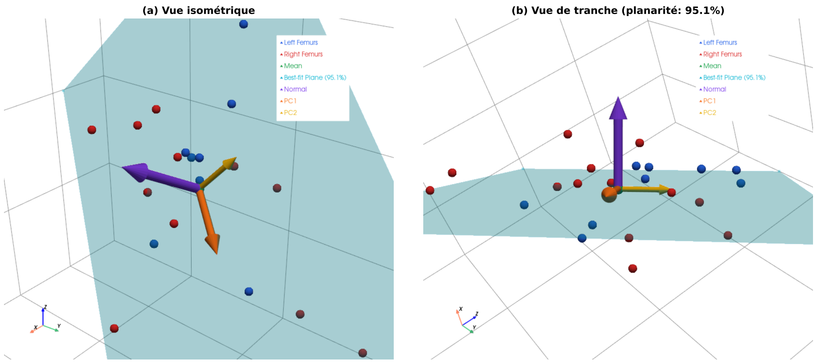


Figure 11 - Latent space representation using the first three principal components. We draw one plane approximating the data distribution.

With this plot, we can see that the data is not uniformly distributed in the latent space. We decide do a PCA on the latent space to see if we can focuse on fewer dimensions.

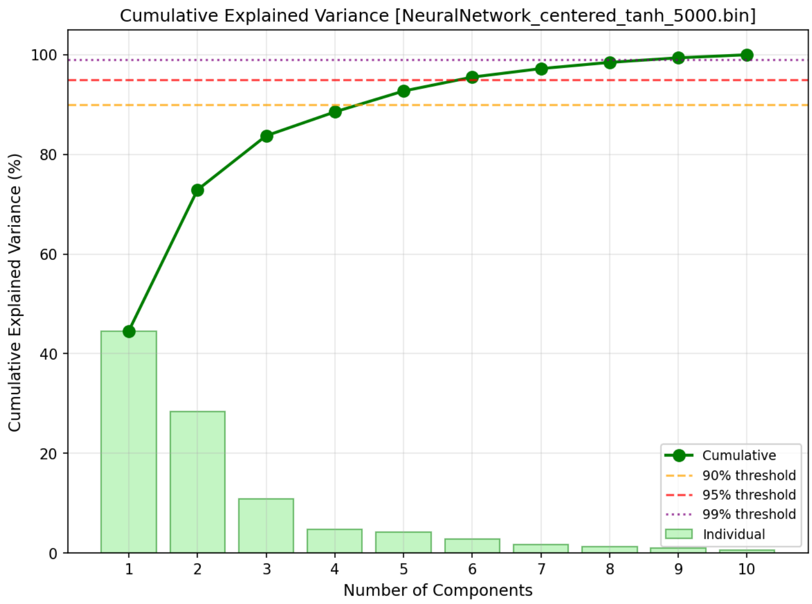
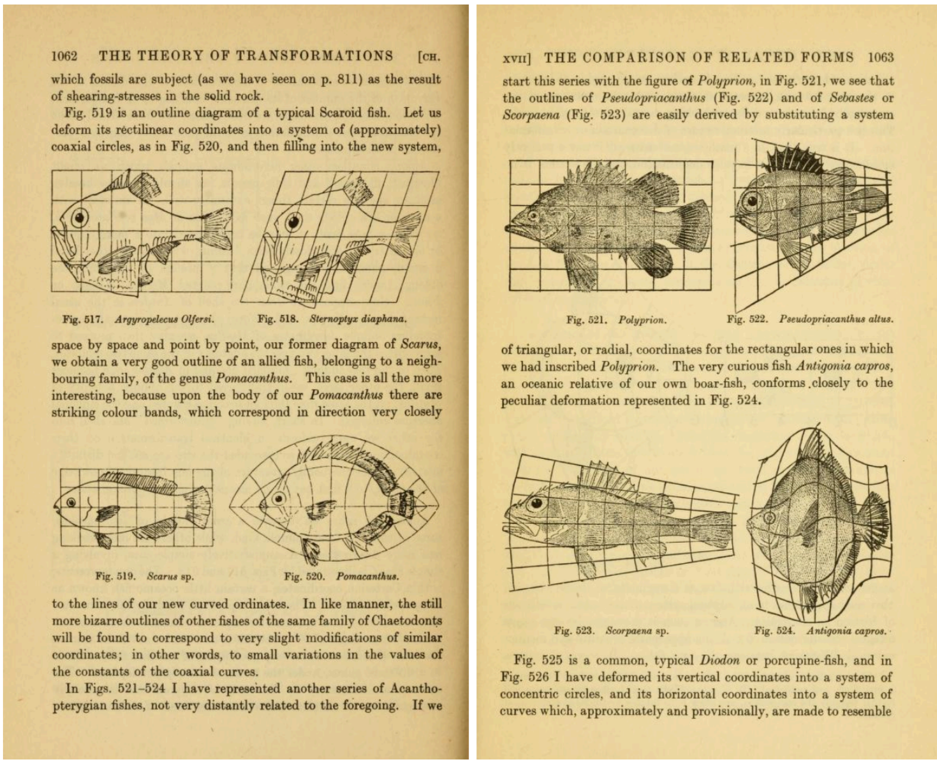


Figure 12 - Cumulative variance explained by PCA on the latent space

With this plot, we can see that the first six principal components explain almost 90% of the variance in the latent space. This means that we can reduce the dimensionality of the latent space from 10 to 6 without losing much information.

4 LDDMM



1062 THE THEORY OF TRANSFORMATIONS [CH.

which fossils are subject (as we have seen on p. 811) as the result of shearing-stresses in the solid rock.

Fig. 519 is an outline diagram of a typical Scaroid fish. Let us deform its rectilinear coordinates into a system of (approximately) coaxial circles, as in Fig. 520, and then filling into the new system,

Fig. 517. *Argyropelecus Olfersi.* Fig. 518. *Sternoptyx diaphana.*

space by space and point by point, our former diagram of *Scarus*, we obtain a very good outline of an allied fish, belonging to a neighbouring family, of the genus *Pomacanthus*. This case is all the more interesting, because upon the body of our *Pomacanthus* there are striking colour bands, which correspond in direction very closely

Fig. 519. *Scarus* sp. Fig. 520. *Pomacanthus.*

to the lines of our new curved ordinates. In like manner, the still more bizarre outlines of other fishes of the same family of Chaetodonts will be found to correspond to very slight modifications of similar coordinates; in other words, to small variations in the values of the constants of the coaxial curves.

In Figs. 521–524 I have represented another series of Acanthopterygian fishes, not very distantly related to the foregoing. If we

XVII] THE COMPARISON OF RELATED FORMS 1063

start this series with the figure of *Polyprion*, in Fig. 521, we see that the outlines of *Pseudopriacanthus* (Fig. 522) and of *Sebastes* or *Scorpaena* (Fig. 523) are easily derived by substituting a system

Fig. 521. *Polyprion.* Fig. 522. *Pseudopriacanthus altus.*

of triangular, or radial, coordinates for the rectangular ones in which we had inscribed *Polyprion*. The very curious fish *Antigonia capros*, an oceanic relative of our own boar-fish, conforms closely to the peculiar deformation represented in Fig. 524.

Fig. 523. *Scorpaena* sp. Fig. 524. *Antigonia capros.*

Fig. 525 is a common, typical *Diodon* or porcupine-fish, and in Fig. 526 I have deformed its vertical coordinates into a system of concentric circles, and its horizontal coordinates into a system of curves which, approximately and provisionally, are made to resemble

Figure 13 -

In an altogether remarkable work, the scholar, biologist, and mathematician D'Arcy Wentworth Thompson (1860-1948) underscored the importance of environmental and physical factors (as opposed to heredity alone) in the morphogenesis of living beings. Since the shape of fish is more or less optimal, there is not an infinite number of distinct "plans," but rather a mere handful of original patterns which allow, through (non-trivial) deformations, the generation of all forms favored by evolution.

To describe the anatomical variability of an observed family or population, it is therefore sufficient to provide a *reference template* (arbitrarily complex, yet common to all observations) and the deformations required to map said template to the individuals. Complexity is thus decoupled into two intelligible components: a *reference image*, complex but fixed; and *deformations* specific to the observed subjects, often simple enough to be described with few parameters.

Remarkable fact: the diagrams above present the variability of fish shapes not as arbitrary displacements of skeletons, but as **coordinate changes, deformations of the ambient space**. It is upon this mathematical concept of *extrinsic* deformation of space (as opposed to *intrinsic* movements of fish particles) that Procrustean analysis and the "LDDMM" theory presented in this chapter rely.

We will attempt to present the big ideas and algorithms of LDDMM for discrete shape spaces with point correspondence (commonly known as landmarks spaces) in an intuitive manner, preserving the most mathematical content as possible but surely making compromises. For full technical details, proofs and rigorous presentation, see [3], [4] , [5], [6] . This study will be centered on the application of LDDMM to human femurs, but the global theory is a general computational anatomy framework.

4.1 Overview

Large Deformation Diffeomorphic Metric Mapping (LDDMM) is a mathematical framework for computing smooth, invertible transformations between shapes [4]. Unlike linear methods that treat shapes as vectors in Euclidean space, LDDMM treats shapes as points on a **Riemannian manifold**, i.e., a curved, locally Euclidean space, respecting the intrinsic geometry of the shape space. Consider the space of all possible femur shapes $\mathcal{S}$. There is no biological nor anatomical reason suggesting this should be a vector space—you cannot simply add two femurs and get a valid femur [3]. In the landmark setting, we model shapes as elements of $\left(\mathbb{R}^3\right)^n$, but the admissible shapes form a nonlinear embedded submanifold $\mathcal{S} \subset \left(\mathbb{R}^3\right)^n$. Let $A = \mathbb{R}^3$ denote the ambient space. Studying large deformations with a Riemannian approach has been an efficient point of view to generate metrics between deformable objects, and to provide accurate, unambiguous and smooth matchings between shapes.

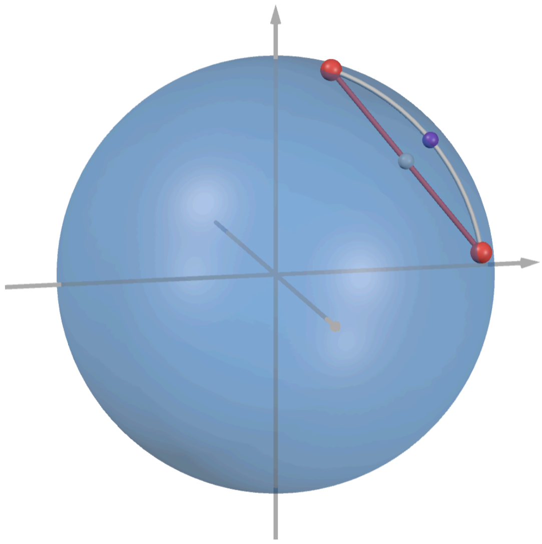


Figure 14 - Curved space is not stable by linear operations, and requires a metric aware of its geometry

In addition to anatomical implausibility of linear transformations, the Euclidean distance poses other challenges to anatomical relevancy, as it treats all point movements equally : this is not the general case in nature, as some deformations physically “cost” more than others, rendering them less probable. The straight line $S_0 \to S_1$ is not the optimal deformation path on the shape manifold.

For statistical shape analysis, we need two fundamental objects:
- A **distance** between shapes (how different are two femurs?)
- A **linear space** for statistics (PCA requires vector operations)

LDDMM provides both through its Riemannian geometry. Crucially, the resulting mean and distances are geometrically meaningful and respect the physical constraints of anatomical deformations [3]. We first present the broad theoretical plan, before diving into technical aspects of the construction aimed at making the problem both interpretable and computationally tractable.

Our goal is to build/learn a relevant metric on $\text{Diff}(A)$, the group of diffeomorphisms (smooth invertible transformations) of the ambient space A. However, since $\text{Diff}(A)$ is curved and infinite dimensional, we cannot perform standard statistical analysis on it. Indeed, there exists a lot of different such ways to transform a shape into another. We thus need to restrict ourselves to some subspace of this group, building a diffeomorphism space that can reflect complex deformations while being computationally viable.

The general framework is as follows: we first construct a vector space (called “velocity field”) whose norm defines the **cost** of an infinitesimal deformation; the diffeomorphisms enabling the deformation of our shapes are then obtained by integrating a flow equation.

This approach is powerful but still yields a bunch of different time-dependent “velocity fields” leading to the same diffeomorphism : we thus focus on the **cheapest** ones w.r.t a general **energy** of the deformation derived from the cost of the velocity field.

One reason for this is it provides a **geodesic distance** on $\mathcal{S}$, computed as the energy of the **cheapest** transformation from S_0 to S_1.

The notion of geodesic is a generalization of the notion of a “straight line” where every step of the movement must lie on the manifold. Here, the “movement” is the deformation, and its belonging to $\mathrm{Diff}(A)$ ensures this property. Geodesics are locally length-minimizing diffeomorphic paths (w.r.t to the energy) between two shapes; we return to the global subtleties later, see [3].

Using this distance, we can define a mean shape of our population of K shapes, commonly referred to as **atlas** in Riemannian geometry.

$$\bar{S} = \arg\min_{S} \sum_{i=1}^{K} d^2(S, S_i). \tag{6}$$

One important result of LDDMM theory is that, given the right diffeomorphism space, focusing on geodesic deformations is not only good for defining a distance :

- It is plausible to believe that nature favors “cheap” deformations from a biological standpoint, analogous to the principle of least action, see [5].
- Since our space of deformations is a manifold, it is locally Euclidean around any shape S, i.e., it is a flat vector space called the **tangent space** at S, denoted $T_S\mathcal{S}$.
- A fundamental theorem on the nature of geodesics establishes a local bijection between geodesic deformations originating from the atlas and their **initial momenta**—vectors with one component attached to every vertex of the shape, which thus live in the $3n$-dimensional cotangent space at the atlas $T^*_{\bar{S}}\mathcal{S}$. Through the cometric, these momenta correspond to initial velocities in the tangent space.

The **initial momentum** P_0 at the atlas encodes “which direction and how far” to travel along a geodesic to reach a target shape. Via the cometric, it corresponds to an initial velocity in the tangent space. This bijection is realized through two fundamental maps:

- The **exponential map** $\mathrm{Exp}_{\bar{S}} : T_{\bar{S}}\mathcal{S} \to \mathcal{S}$ sends an initial velocity to the shape reached by following the corresponding geodesic for unit time.
- The **logarithm map** $\mathrm{Log}_{\bar{S}} : \mathcal{S} \to T_{\bar{S}}\mathcal{S}$ is its inverse, returning the initial velocity that generates a geodesic to a given shape.

Said in a more compact way, the following diagram commutes :

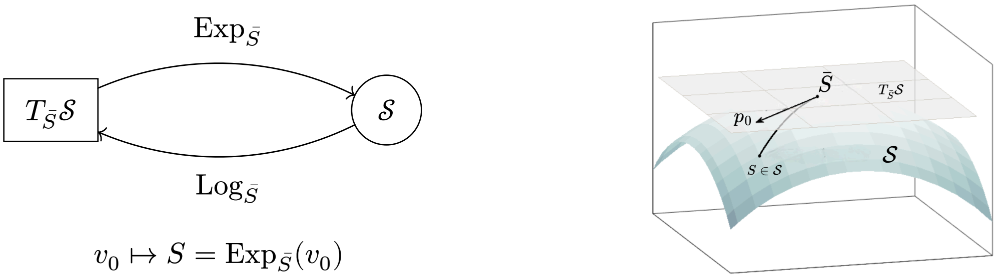


Figure 15 - The exponential and logarithm maps establish a local diffeomorphism between the tangent space at the atlas and the shape manifold.

Beware this is only a **local** result, thus only valid for “small” deformations of the atlas, a limitation we clarify and address later.

This geodesic point of view thus yields us the two fundamental objects needed for statistical analysis. We can then perform standard PCA (linear or kernel) on $T_{\bar{S}}\mathcal{S}$ in order to understand the most probable deformations of the mean femur $\bar{S}$. We call this **Tangent PCA** or **Principal Geodesic Analysis** (PGA).

Computationally, the most frequent operations to perform are Exp (called **geodesic shooting**) and Log (called **registration**). Rendering these computations tractable is thus one important goal that influences the construction of the diffeomorphism space.

4.2 Diffeomorphism spaces in LDDMM : General Principle

In this section, we describe the general principle of the building of diffeomorphism spaces in the LDDMM framework. This presentation is adapted from [3], which the interested reader should refer to for a more detailed presentation.

4.2.1 General framework : integrating the flow of vector fields

Definition. A vector space V of vector fields on $\mathbb{R}^d$ is said to be *admissible* if it satisfies the following conditions.

1. V is a Hilbert space. We denote its norm by $\|\cdot\|_V$ and its inner product by $\langle \cdot, \cdot \rangle_V$.
2. $(V, \|\cdot\|_V)$ is continuously embedded in $\left(C_0^1\left(\mathbb{R}^d, \mathbb{R}^d\right), \|\cdot\|_{1,\infty}\right)$, the space of C^1 fields on $\mathbb{R}^d$ vanishing at infinity, along with their partial derivatives. There exists, therefore, a constant $c_V > 0$ such that:

$$\forall v \in V, \quad \|v\|_{1,\infty} \leq c_V \|v\|_V. \tag{7}$$

The diffeomorphisms we are about to define are viewed as solutions to a flow equation. We observe a vector field v_t (modeling infinitesimal deformations) deforming our space over time. Upon reaching time t, the deformation defines a diffeomorphism. A theorem guarantees the existence and uniqueness of such solutions. However, we require additional hypotheses on our vector fields, specifically an L^2 control.[1]

[1]We actually only need a L^1 control for the theorem, but a lemma allows us to restrict our following study to vector fields with an L^2 control whilst not losing any generality.

Definition. We define the following space and norm.

$$L^2_V = L^2([0,1], V) \text{ endowed with } \|v\|_{L^2_V} = \sqrt{\int_0^1 \|v_t\|_V^2 \, \mathrm{d}t}. \tag{8}$$

The elements of L^2_V are time dependent vector fields $v : t \mapsto v(t)$ where we denote $v(t) = v_t$, for $t \in [0,1]$ and $v_t \in V$.

Theorem. Let $v \in L^2_V$. For all $x \in \mathbb{R}^d$, there exists a unique continuous mapping $t \mapsto \Phi_t^{v(x)}$ from $[0,1]$ to $\mathbb{R}^d$ satisfying the flow equation.

$$\Phi_t^v(x) = x + \int_0^t v_s \circ \Phi_s^v(x) \, \mathrm{d}s. \tag{9}$$

Alternatively,

$$\Phi_0^v(x) = x \quad \text{and} \quad \frac{\partial \Phi_t^v}{\partial t}(x) = v_t(\Phi_t^v(x)). \tag{10}$$

The diffeomorphisms that will serve our purpose are thus the elements of the set:

$$\mathcal{D}_V = \left\{\Phi_1^v \mid v \in L^2_V\right\}. \tag{11}$$

Indeed, it suffices to consider flows at time 1 via time renormalization. This definition of diffeomorphisms on our space allows us to establish a metric, which in turn will allow us to evaluate the cost associated with using a diffeomorphism to deform our space.

Proposition. $\mathcal{D}_V$ is a group and a complete space for the metric. $d_V(\mathrm{id}, \Phi) = \inf\left\{\|v\|_{L^2_V} \mid v \in L^2_V, \Phi_1^v = \Phi\right\}$. Which can be extended by right-invariance:

$$d_V(\Phi, \Psi) = d_V\left(\mathrm{id}, \Psi \circ \Phi^{-1}\right). \tag{12}$$

This metric is simply defined as the infimum of the norm of vector fields that deform the identity to $\Phi(\mathrm{id})$.

If we take two diffeomorphisms in $\mathcal{D}_V$, there exists a geodesic between them for the metric we have defined, i.e., the infimum is actually a minimum.

Proposition. Let $\Phi, \Psi \in \mathcal{D}_V$. There exists $v \in L^2_V$ such that $\Phi_1^v = \Phi \circ \Psi^{-1}$ and $d_{V(\Phi,\Psi)} = \|v\|_{L^2_V}$.

We say that $\mathcal{D}_V$ is the group of diffeomorphisms modeled on V starting from the identity.
In the “limit” case where V is the space of constant fields, identified with $\mathbb{R}^m$ equipped with the Euclidean metric, $\mathcal{D}_V$ is simply the space of translations equipped with the natural Euclidean distance.

By considering a more flexible space V, we cause the number of degrees of freedom—and thus the complexity of the space of diffeomorphisms $\mathcal{D}_V$—to explode. The entire objective of this exposition will be to see how to choose V to keep the problem *reasonable* and numerically solvable.

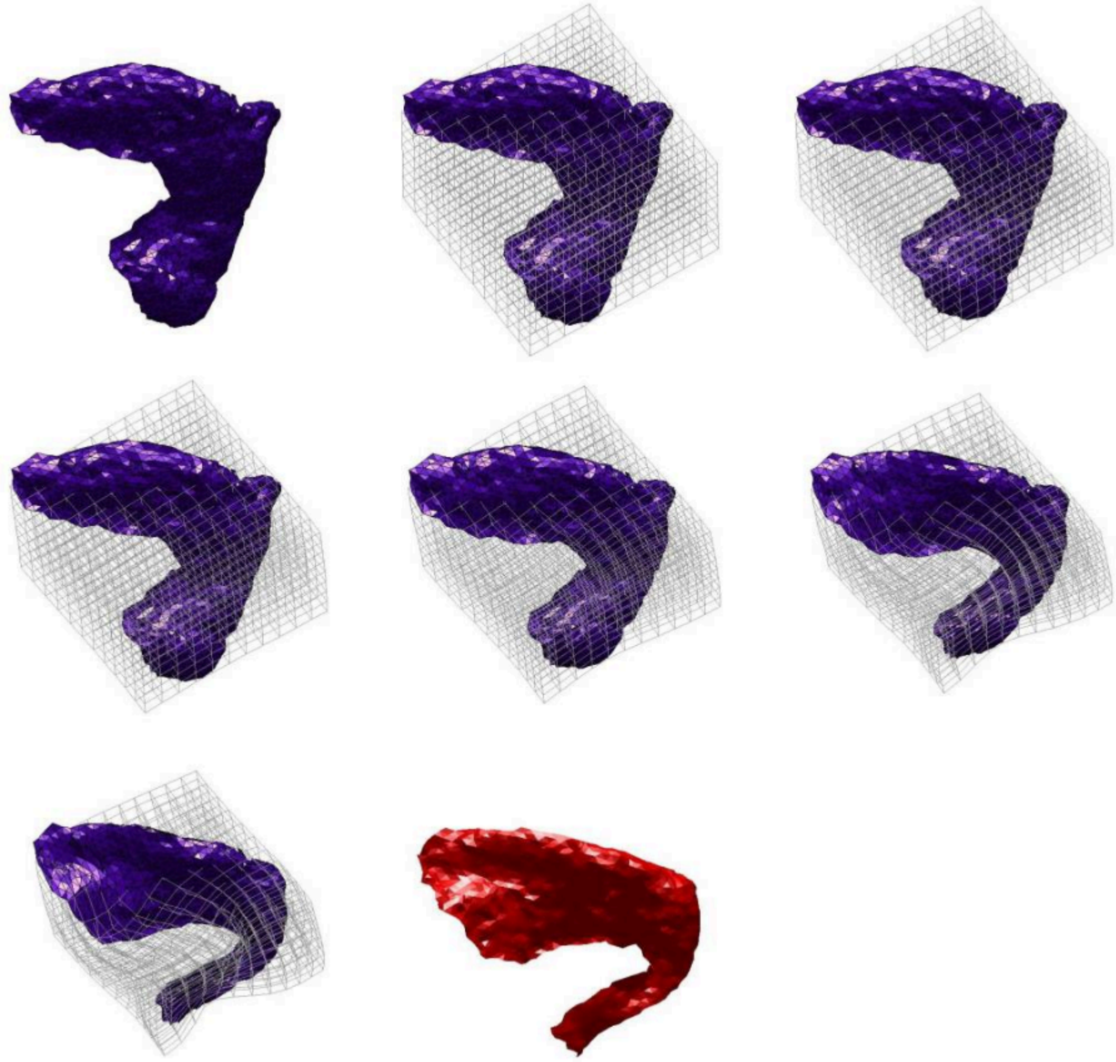


Figure 16 - Deformation of a surface in another under the action of a diffeomorphism. Step by step, we see the ambient grid deform under the action of a vector field v_t.

4.3 The Matching problem

We have defined a metric on a general diffeomorphism space that deforms our shape space $\mathcal{S}$. Considering arbitrary shapes $S_0, S_1 \in \mathcal{S}$, a theorem guarantees the existence of a diffeomorphism $\Phi \in \mathcal{D}_V$ such that $\Phi(S_0) = S_1$. We could thus theoretically define a distance on $\mathcal{S}$ in the following way:

$$d_V(S_0, S_1) = d_V(\mathrm{id}, \Phi) = \min_{v \in L^2_V} \sqrt{\int_0^1 \|v_t\|_V^2 \,\mathrm{d}t} \tag{13}$$

.

In practice, finding the Φ that exactly maps S_0 to S_1 is too hard, we thus relax the problem by defining a naive **matching** distance, which we can do because our shape space is a landmark space.[2]

[2]This becomes way harder in non-landmark spaces, and is done via Varifolds. See [3] for more details.

Matching distance. Let $\Phi \in \mathcal{D}_V$, $S_0 = \{x_i, i \in [\![1, n]\!]\} \in \mathcal{S}$ and $S_1 = \{z_i, i \in [\![1, n]\!]\} \in \mathcal{S}$. The matching distance between S_1 and $\Phi(S_0) = \{y_i, i \in [\![1, n]\!]\}$ is defined by:

$$d_{\mathcal{S}}(\Phi) = \|\Phi(S_0) - S_1\|_2^2 = \sum_{i=1}^{n} |y_i - z_i|^2. \tag{14}$$

This matching (L^2, squared euclidean) distance is not a relevant quantity in general as already discussed, but can be used to assess if two shapes are “very” close or not.

We are now ready to properly define a relevant **energy** (cost) of the transformation from S_0 to S_1, by minimizing the cost of Φ and relaxing the matching :

Matching problem. Let $\lambda > 0$. The matching problem is defined by.

$$\text{Minimize} \qquad E(\Phi) = \lambda d_V(\mathrm{id}, \Phi) + d_{\mathcal{S}}(\Phi) \text{ over } \mathcal{D}_V. \tag{15}$$

Which is equivalent to minimizing the following functional over L^2_V :

$$E(v) = \lambda \int_0^1 \|v_t\|_V^2 \,\mathrm{d}t + d_{\mathcal{S}}(\Phi_1^v). \tag{16}$$

To solve this matching problem, we have an existence theorem at our disposal, which assures us that this endeavor is not futile and that there effectively exist diffeomorphisms allowing us, in practice, to map one shape to another in an optimal manner.

In practice, the task consists of finding a diffeomorphism Φ that deforms a shape S_0 into a shape $\Phi(S_0)$ close to S_1. The optimal shape $\Phi(S_0)$ can then be viewed as a *barycenter* of the points S_0 — with weight λ — and S_1 — with weight 1.

The real number λ is a *regularization* weight, which compels $\Phi(S_0)$ to remain within a neighborhood of S_0: one often selects $0 < \lambda \ll 1$, so as to obtain a model $\Phi(S_0)$ close to the observation S_1, yet remaining at a “finite” deformation distance from the shape S_0.

4.4 Kernel Methods : Efficient and Interpretable Vector Fields

The matching problem as expressed in is over L^2_V, which can *a priori* be infinite dimensional. Our goal is thus to build V such that we can *reduce*[3] the problem to a finite dimensional setting, making it numerically solvable. In this section, we define V as the **Reproducing Kernel Hilbert Space** (RKHS) induced by a positive definite kernel k. In this context, we perform a first reduction step allowing us to only focus on defining the deformation at the landmarks. We then introduce the **Gaussian kernel**, used in practice. Finally, we reinterpret our matching problem in terms of **Riemannian geometry**, ultimately yielding a *stunning* reduction of the problem to **initial momenta** in $\mathbb{R}^{3n}$.

[3] *Reduction* is the process of proving that the optimization of a functional over a larger space can be done over a smaller space

4.4.1 Reproducing space and kernels

In the following section, $A = \mathbb{R}^3$ is our ambient space, thus vectors of dimension n of A represent shapes with n landmarks. We adopt the notation q for positions (landmarks) in A and p for momenta or vectors, in order to anticipate the Hamiltonian formalism introduced later.

> **RKHS.** Let H a Hilbert space of vector fields $A \to \mathbb{R}^3$. H is called a *Reproducing Kernel Hilbert Space* (or RKHS) when the linear functionals
> $\delta_q^p : f \in H \mapsto f(q) \cdot p$, where $q \in A$ and $p \in \mathbb{R}^3$, are continuous.

We can then apply the Riesz theorem to δ_q^p and define the reproducing kernel of an RKHS:

> **Reproducing kernel.** Let H be an RKHS. The *reproducing kernel* of H is the mapping $k_H : A^2 \to \mathcal{L}(\mathbb{R}^3)$ such that:
>
> $$\forall q \in A, k_H(q, \cdot)p = K_H \delta_q^p \tag{17}$$
>
> meaning that $k_H(q, \cdot)p$ is the unique element of H such that:
>
> $$\forall h \in H, \delta_q^p(h) = \langle k_H(q, \cdot)p, h\rangle_H = h(q) \cdot p \tag{18}$$

> **Positive kernel.** Let $k : A^2 \to \mathcal{L}(\mathbb{R}^3)$. For any shape $S = (q_1, ..., q_n) \in A^n$, we define the associated kernel matrix K_S as the following square block matrix of size $3n$:
>
> $$K_S = \begin{pmatrix} k(q_1, q_1) & k(q_1, q_2) & ... & k(q_1, q_n) \\ k(q_2, q_1) & k(q_2, q_2) & ... & k(q_2, q_n) \\ \vdots & \vdots & \ddots & \vdots \\ k(q_n, q_1) & k(q_n, q_2) & ... & k(q_n, q_n) \end{pmatrix}. \tag{19}$$
>
> where each entry $k(q_i, q_j)$ is a 3×3 matrix. The map k is said to be a **positive kernel** if for every stacked vector of momenta $P = (p_1, ..., p_n)^T \in (\mathbb{R}^3)^n$, the quadratic form defined by K_S is non-negative:
>
> $$P^T K_S P \geq 0 \Leftrightarrow \sum_{i,j} (p_i)^T k(q_i, q_j) p_j \geq 0. \tag{20}$$

One can show the equivalence between the notions of reproducing kernel and positive kernel.

> **Theorem.**
>
> 1. The reproducing kernel of an RKHS is a positive kernel.
> 2. For any positive kernel k on A , there exists a unique RKHS included in $(\mathbb{R}^3)^A$ whose reproducing kernel is k.

The main appeal of reproducing kernels is they provide a simple method for defining admissible vector field spaces that are interpretable and algorithmically efficient. We first choose the positive kernel k then define V as the unique RKHS corresponding to k.

4.4.2 Gaussian Kernel

4.5 Spatial Reduction of the Matching problem

Consider a fixed time $t \in [0, 1]$, and suppose the current positions of our n landmarks forming the shape S are $q_1, ..., q_n$.
Physically, the term $k(x, q_i)p_i$ represents the velocity field generated in the entire space by a single "push" (momentum p_i) applied at the point q_i.

- If we push on a landmark q_i, the kernel acts as a transmission function, dragging the surrounding space along with it.
- The "shape" of this drag is determined by the kernel function.

Therefore, the subspace spanned by these kernels:

$$V_S = \operatorname{span}(\{k(\cdot, q_i)p_i \mid i = 1..n, p_i \in \mathbb{R}^3\}). \tag{21}$$

represents the set of all infinitesimal deformations that can be constructed **solely** by pushing on the landmarks at time t.

We recall that at each time step, our goal is to find the vector field v_t that moves our landmarks q_i while minimizing the infinitesimal deformation energy $\|v_t\|_V^2$.[4]

Consider any candidate vector field $v_t \in V$. We can decompose it into two orthogonal components:

$$v_t = v_t^S + v_t^\perp. \tag{22}$$

where $v_t^S \in V_S$ is built from the kernels applied at the landmarks, and $v_t^\perp$ is orthogonal to them.

The crucial insight comes from the **Reproducing Property** of the RKHS. For any momentum p, the inner product with the kernel evaluates the field at the landmark:

$$\langle v_t^\perp, k(\cdot, q_i)p \rangle_V = p \cdot v_t^\perp(q_i). \tag{23}$$

Since $v_t^\perp$ is orthogonal to the kernel, it is orthogonal to the landmark, i.e., $v_t^\perp(q_i) = 0$.

The component $v_t^\perp$ does not help move the landmarks, yet still adds to the total cost $\|v_t\|_V^2 = \|v_t^S\|_V^2 + \|v_t^\perp\|_V^2$. To minimize cost, we must therefore set $v_t^\perp = 0$.

This reasoning tells us that the optimal deformation of the whole space is completely determined by the interaction of n "bumps" centered at our landmarks, which yields a powerful reduction of our matching problem.

> **Theorem.** Let $S = \{q_1, ..., q_n\}$ denote the current landmarks. The vector field $v_t \in V$ which interpolates specific velocities $v_{t(q_i)}$ with minimal norm is a linear combination of the kernel centered at the landmarks:
>
> $$\forall x \in A, \quad v_{t(x)} = \sum_{i=1}^{n} k(x, q_i)p_i. \tag{24}$$
>
> where the vectors $p_i \in \mathbb{R}^3$ act as **momenta** determining the strength and direction of the local deformation.

[4]Indeed, this implies a minimization of the total energy $\int_0^1 \|v_t\|_V^2 \, dt$.

Crucially, the finite dimensional nature of $L^2_{V,S}$ makes for a simple computation of the norm :

$$\|v_t\|_V^2 = \sum_{i,j \in [\![1,n]\!]} p_j^T k(q_i, q_j) p_i. \tag{25}$$

Corollary. The optimal solution of the problem

$$\text{Minimize} \quad E(v) = \lambda \int_0^1 \|v_t\|_V^2 \, \mathrm{d}t + d_S(\Phi_1^v). \tag{26}$$

can be searched over $L^2_{V,S} = \left\{ v \in L^2_V, \forall t \in [0,1], v_t \in V_{\Phi_t^v(S)} \right\}$, space of dimension $3n$.

At each time step, instead of solving a PDE for a complex function $v(x)$, we have reduced the problem to finding the optimal momenta p_i for each landmark. The global flow is simply the superposition of these local kernel influences. More precisely, for any arbitrary point x in our ambient space A (e.g., a point of a femur mesh), the flow equation becomes:

$$\frac{\partial \Phi_t^v}{\partial t}(x) = v_t(\Phi_t^v(x)) = \sum_{j=1}^n k(\Phi_t^v(x), q_j) p_j. \tag{27}$$

This shows that the deformation of the entire space is "interpolated" from the momenta p_j of the driving landmarks.

If we track the landmarks themselves, denoting $q_{i(t)} = \Phi_t^v(q_i(0))$ and $S(t) = (q_1(t), ..., q_n(t))$. Defining the stacked momentum $P(t) = (p_1(t), ..., p_n(t))^T$, the landmark dynamics can be written compactly as:

$$\frac{\partial S}{\partial t} = K_{S(t)} P(t). \tag{28}$$

This important reduction step has made the problem somewhat[5] computationally tractable since we are optimizing over a finite set of parameters, we can compute $d_S(\Phi_1^v)$ by integrating the flow equation, and easily compute $\|v_t\|_V^2$, hence $E(v)$ as a whole.
However, a Riemannian geometric view of our dynamical system will shed light on the true nature of our problem, leading us to the most significant and *beautiful* reduction step.

4.6 Riemannian Geometry of Landmarks

4.6.1 Introduction

In this section, we intuitively introduce the fundamental concepts of Riemannian Geometry, the study of smooth curved spaces called **manifolds**. For a comprehensive introduction to this field of study, see [3].

It is an abstract model that captures the essence of all possible spaces having the same intrinsic geometry. By the "essence" we mean knowledge of the distance between any two points, for this and this alone determines the intrinsic geometry. In fact, it is sufficient to have a rule for the infinitesimal distance between neighbouring points. This rule is called the **metric**. Given this metric, we may determine the length of any curve as an infinite sum (i.e., integral) of the infinitesimal segments

[5]This is really theoretical, in the sense that you can write algorithms that perform this optimization. In practice, this problem is still way too complex to be solved in a reasonable amount of time.

into which it may be divided. We can then define a distance using the shortest paths between points on the manifold. These shortest paths on a curved space are the equivalent of straight lines in the plane, they are called **geodesics**. Thus, to use this new word, we may say that geodesics in Euclidean space are straight lines, and geodesics on the sphere are great circles.

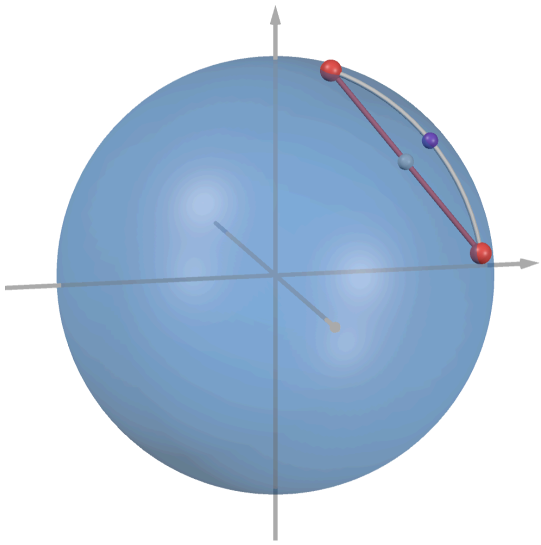


Figure 17 - Euclidean distance and distance on the sphere as a Riemannian manifold

However, this *length-minimizing* definition of geodesics is subtle, because even on the sphere, we see that nonantipodal points are connected by two arcs of the great circle passing through them: the short one (which is the shortest route) and the long one. Yet the long arc is every bit as much *straight*, thus a geodesic, as the short one. We thus provide a purely *local* characterization of geodesics.

One key property of manifolds is they are locally Euclidean, i.e., look like flat space around any point when *zooming* enough. This is formalized by the notion of **tangent space**, the flat space hugging our manifold exactly at a point.

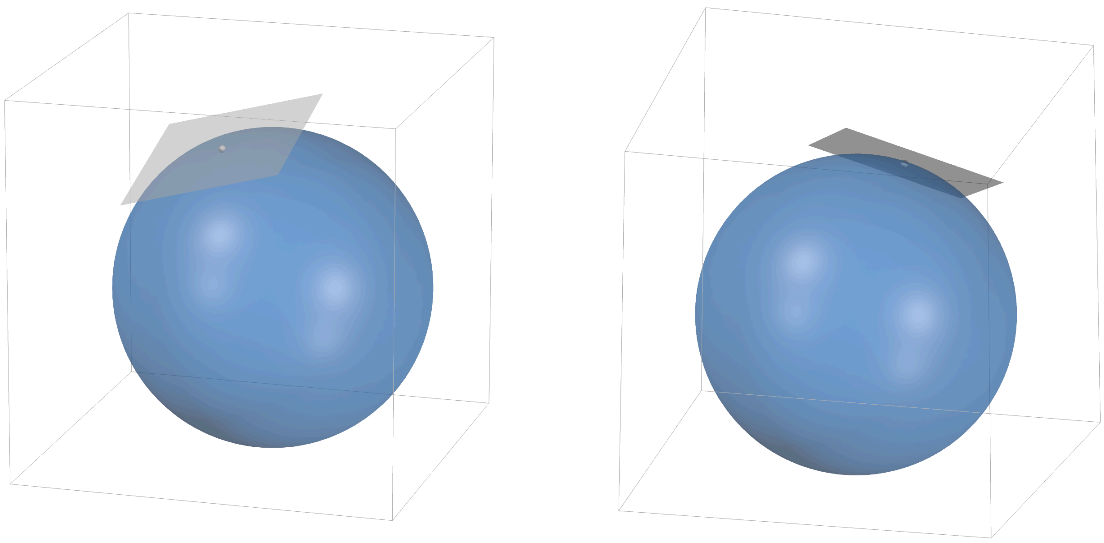


Figure 18 - The tangent space at a point of the sphere is the plane touching the sphere at that single point, containing all possible directions in which one can instantaneously move starting from there.

We experience this everyday walking on our beautiful planet Earth. To us, going "straight" is equivalent to the definition in flat space, i.e., walking without ever *turning*. As we live **on** the surface of Earth, thus do not see its curvature, pursuing this local straight behavior yields a global geodesic. The only parameter that determines the particular geodesic we trace and the time it takes is our initial **velocity** vector. Indeed, once we are set on our path, flowing on the surface following the local rule of not turning is enough to characterize a unique geodesic. Conversely, provided we stay close enough to the source point, the unique geodesic connecting given source and target points is fully characterized by the initial velocity starting from the source.

This leads to another interpretation of the tangent space. Consider (conveniently) a Riemannian manifold $\mathcal{S}$ and a point $\bar{S}$. The tangent space at $\bar{S}$, denoted $T_{\bar{S}}\mathcal{S}$ is the space of all possible velocity vectors a being moving **on** the surface can follow when it is in $\bar{S}$. Armed with this insight, one can formalize the velocity $\leftrightarrow$ geodesic relationship.

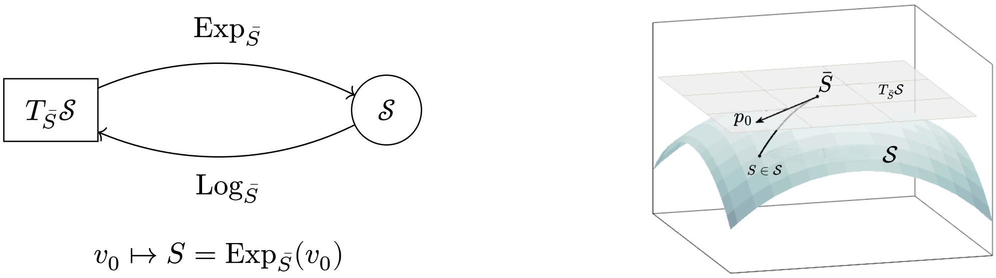


Figure 19 - The exponential and logarithm maps establish a local diffeomorphism between the tangent space at a point and the manifold.

- The **exponential map** $\mathrm{Exp}_{\bar{S}} : T_{\bar{S}}\mathcal{S} \to \mathcal{S}$ sends an initial velocity vector to the point reached by following the corresponding geodesic for unit time.
- The **logarithm map** $\mathrm{Log}_{\bar{S}} : \mathcal{S} \to T_{\bar{S}}\mathcal{S}$ is its inverse, returning the initial velocity vector that generates a geodesic to a given point.

As shown earlier by the example of antipodal points on the sphere, this result is only true **locally** : if the target and source points are too far apart, there can be multiple geodesics connecting them, thus multiple potential velocity vectors, yielding a multivalued Log map.

4.6.2 The Landmarks Manifold

We interpret the set of all possible landmark configurations, i.e., our shape space $\mathcal{S}$ as a smooth manifold embedded in $(\mathbb{R}^3)^n$. To define the geometry, we must distinguish between the deformation itself and the force generating it. Let $S \in \mathcal{S}$ be a shape state (which we later interpret as the shape attained at time $t \in [0, 1]$ of a trajectory in $\mathcal{S}$).

The tangent space $T_S\mathcal{S}$ is the space of all possible instantaneous deformations of the shape S. An element $v_t \in T_S\mathcal{S}$ is a vector field restricted to the landmarks, i.e., a collection of velocity vectors attached to each point, which we can think of as a global velocity vector attached to the shape:

$$v_t = (v_{t,1}, ..., v_{t,n}) \in (\mathbb{R}^3)^n. \tag{29}$$

This is exactly the definition on V_S in , which is the infinitesimal deformation space we are optimizing over thanks to our last reduction step.

The **Cotangent Space** $T_S^*\mathcal{S}$ is the dual space of the tangent space. It contains the linear forms acting on velocities. An element $P(t) \in T_S^*\mathcal{S}$ is called the **momentum** (or co-vector). In our context, it represents the "force" or "constraints" applied to the landmarks to drive the deformation. Like velocity, it is numerically represented as a stacked vector in $\left(\mathbb{R}^3\right)^n$.

In standard Euclidean space, velocity and momentum are often identified ($v_t = P(t)$). In a curved space, they are distinct, and linked by the physics / metric / curvature of the medium.

At first, it seems unintuitive to introduce this space and not do everything in the classical tangent space. In practice, the kernel's impact is best interpreted with regards to momentum.

The kernel matrix K_S, defined block-wise by $K_{i,j} = k\left(q_i, q_j\right)$, acts as the Cometric (inverse metric) on the manifold. It maps momentum to velocity:

$$v_t = K_S P(t) \quad \Longleftrightarrow \quad v_{t,i} = \sum_{j=1}^{n} k\left(q_i, q_j\right) p_{j(t)}. \tag{30}$$

- The momentum P is the control variable (the input "push").
- The kernel K_S is the transmission mechanism (smoothing/correlation).
- The velocity v_t is the resulting deformation (the output).

Consequently, the Riemannian metric g_S, defined on $\left(T_S\mathcal{S}\right)^2$, which measures the cost of the deformation is expressed as[6] :

$$g_{S(v_t,v_t)} = \|v_t\|_V^2 = P(t)^T K_S P(t) = v_t K_S^{-1} v_t \tag{31}$$

4.7 Reduction of the Matching Problem to initial momentum

In this section, we leverage the Riemannian structure of the Landmarks shape space to interpret the Matching problem as a geodesic problem. Then, by exploiting the conservation of energy principle on geodesics, an idea originating from classical mechanics, we reduce the problem to the space of initial momentum $T_S^*\mathcal{S} \simeq \mathbb{R}^{3n}$.

4.7.1 Recap on the current state of the problem

We have established that our shape space is a Riemannian manifold equipped with a cometric K_S. We can now reinterpret our original optimization problem through this geometrical lens. We denote S_0, S_1 respectively our source and target shapes. Recall that we are minimizing the energy functional over $L^2_{V,S}$:

$$E(v) = \lambda \int_0^1 \|v_t\|_V^2 \,\mathrm{d}t + d_{\mathcal{S}}(\Phi_1^v). \tag{32}$$

Using our Riemannian formalism, the integral term is exactly the cost / length of the trajectory $S(t)$ on the manifold, for $t \in [0,1]$:

$$L_{\Phi_1^v}^{S_0} = \int_0^1 P(t)^T K_{\Phi_t^v(S)} P(t) \,\mathrm{d}t. \tag{33}$$

[6]The metric is actually an inner product on the tangent space, thus expressed as $g_{S(u,v)} = u^T M_S v$ with M_S a matrix. The derivation implicitly done here is $M_S = K_S^{-1}$, which exists by strict positivity of the kernel. Our focus is on using the metric to measure a norm.

Where we recall that $\Phi_t^v(q_i(0)) = q_i(t)$, thus $S(t) = (q_1(t), ..., q_n(t))$ is the shape deformed after time t by the diffeomorphism obtained by integrating the time-dependent vector field v up to t.

4.7.2 Matching Problem as a Geodesic

In the following section, we consider the exact-matching formulation (or the inexact problem with a hard constraint $\Phi_1^v(S_0) = S_1$). In this setting, $d_{\mathcal{S}}(\Phi_1^v) = 0 \Leftrightarrow \Phi_1^v(S_0) = S_1$, i.e., the source shape is exactly matched to the target shape. Consequently, it is also a minimizer of $L_{\Phi_1^v}^{S_0}$.

We interpret this geometrically by the fact that $\Phi_t^v(S_0)$ is a *length-minimizing* curve on $\mathcal{S}$ between S_0 and S_1. By definition, this optimal deformation $S_0 \longrightarrow S_1$ is thus a **geodesic** of $\mathcal{S}$.

As mentioned earlier, geodesics (defined as locally straight curves) are in general, not unique. Our optimal trajectory is a length-minimizing geodesic for the chosen formulation.

4.7.3 Hamiltonian Dynamics and Conservation of Energy

We have established that our optimal deformation is a geodesic. Earlier , we have argued that, intuitively, given source and target, the only parameter that determines a geodesic and the time it takes is the initial **velocity** vector.

This is nothing more than a **classical mechanics** argument. We can view the evolving shape as a particle moving through a curved space, governed by physical laws. The state of such a system is described by its **Total Energy** defined as the sum of Kinetic energy (K_E) and Potential energy (P_E):

$$\text{Total Energy (H)} = \underbrace{\text{Kinetic Energy } (K_E)}_{\text{Motion / Inertia}} + \underbrace{\text{Potential Energy } (P_E)}_{\text{External Forces}}. \tag{34}$$

In our specific case, no external forces apply to the deformation hence the absence of potential energy.
The shape thus behaves as a free particle: it “coasts” along the manifold solely due to its initial inertia, thus following a geodesic (equivalent to “not turning” in our previous walking analogy). A fundamental law of mechanics is the **Conservation of Total Energy** for isolated systems. Since $P_E = 0$, this implies the **Conservation of Kinetic Energy**.

The ubiquity of Riemannian geometry in the description of physical systems allows us to formalize this via a theorem true for every Riemannian manifold, originally emanating from the **Hamiltonian** formulation of classical mechanics.

We define the **Hamiltonian** for every shape S, momentum P and associated velocity $v^P \in V$ as:

$$H(S, P) = \frac{1}{2} P^T K_S P = \frac{1}{2} \left\| v^P \right\|_V^2. \tag{35}$$

Geodesic equations. Any geodesic trajectory $(S(t), P(t))$ on our manifold satisfies the canonical system:

$$\begin{cases} \frac{\partial S}{\partial t} = \partial_P H(S, P) = K_S P \\ \frac{\partial P}{\partial t} = -\partial_S H(S, P) = -\frac{1}{2} \nabla_S (P^T K_S P) \end{cases}. \tag{36}$$

We have already encountered the first equation : it is exactly the flow equation for the landmarks we derived using RKHS theory. This is not surprising : they are both interpretations of the same trajectory $S(t)$.

The second equation, however, is completely new to us : it tells us how the momentum (thus velocity) should evolve. It arises specifically when interpreting our optimal deformation as following a geodesic.

A simple application of the chain rule combined with the substitutions of both equation then yields the following result.

Conservation of Energy. Let $(S(t), P(t))$ be a solution to the canonical system. We have :

$$\frac{\mathrm{d}}{\mathrm{d}t} H(S(t), P(t)) = 0 \Rightarrow H(S(t), P(t)) = H(S_0, P_0). \tag{37}$$

The total energy is conserved.

Applying the theorem to our hamiltonian containing only kinetic energy, we get :

$$\int_0^1 \|v_t\|_V^2 \,\mathrm{d}t = \int_0^1 2H(S_0, P_0) \,\mathrm{d}t = 2H(S_0, P_0) = P_0^T K_{S_0} P_0. \tag{38}$$

The entire trajectory $S(t)$ is fully determined by its initial momentum $P_0 = (p_1, ..., p_n)^T \in (\mathbb{R}^3)^n$.

4.7.4 Matching Problem over initial momentum space

Matching Problem over momentum space. Let $\lambda > 0$. The Matching problem over $L^2_{V,S}$.

$$\text{Minimize} \quad E(v) = \lambda \int_0^1 \|v_t\|_V^2 \,\mathrm{d}t + d_S(\Phi_1^v). \tag{39}$$

Is equivalent to the following problem over $\mathbb{R}^{3n}$:

$$\text{Minimize} \quad E(P_0) = \lambda P_0^T K_S P_0 + d_S\left(\Phi_1^{v_{P_0}}\right). \tag{40}$$

Starting from an infinite dimensional setting over L^2_V, choosing V to be a RKHS space allowed us to reduce the problem to a finite dimensional problem over the time interval $[0, 1]$. Finally, interpreting our problem as finding a geodesic in a Riemannian manifold reduced the problem to finding the optimal initial condition P_0, which has the dimension of our shape.

5 Discussion

We compared PCA and neural networks to model the 3D variability of femur shapes. PCA efficiently reduces dimensionality and captures the main linear modes, but struggles with non-linear anatomical variations. The neural network autoencoder overcomes this by learning non-linear features, leading to better reconstructions and the ability to generate new, plausible femurs.

Exploring the latent space of the neural network, we found that most of the variance is explained by a few directions: the first six principal components account for almost 90% of the variability. This shows that the network compresses the essential information into a compact, structured subspace.

Our Python tools for visualization, mesh comparison, and latent space exploration were crucial for debugging and evaluating the models. They revealed that, while the neural network can generate a wide range of shapes, some are not anatomically realistic—highlighting the need for more constraints or regularization.

In short, combining PCA and neural networks gives a flexible and powerful framework for shape modeling. The neural network's ability to capture non-linear variations and generate new shapes is a significant advantage, but ensuring anatomical plausibility remains a challenge for future work.

6 Conclusion and future work

In this work, we showed that both PCA and neural networks are effective for reducing the dimensionality of femur shapes. PCA provides a compact linear description of the main modes of variation, while the neural network captures non-linear features and is able to generate new, plausible femur shapes. Analysis of the latent space confirms that the essential information can be represented in a low-dimensional, structured way.

To improve our results, several directions are possible.

One possible improvement is to augment the dataset using PCA-based generation. While this does not add new information, since it only creates linear combinations of existing data, it can still help the neural network generalize better.

Another direction is to enhance the neural network architecture and training process, for example by increasing the depth of the network. This could allow the network to learn more complex features, but would require careful tuning to avoid overfitting and also the training process will be longer.

We can also improve the training time of the neural network by implementing more advanced multi-threading techniques, such as a thread pool (see Section 3.2.1.3).

The network could also be used to do clustering on healty and unhealthy femurs, by training it on a larger dataset containing both types of femurs. This would allow us to better understand the differences between these two categories and potentially identify pathological shapes.

Finally, exploring more advanced generative models, such as Variational Autoencoders, could provide better control over the generation process and improve the realism of the generated femur shapes. These models can learn more complex distributions and might help in generating anatomically plausible shapes.

Bibliography

[1] "OpenMP." [Online]. Available: https://www.openmp.org/

[2] "PyVista." [Online]. Available: https://docs.pyvista.org/index.html

[3] L. Younes, *Shapes and Diffeomorphisms*, vol. 171. in Applied Mathematical Sciences, vol. 171. Springer, 2010. doi: 10.1007/978-3-642-12055-8.

[4] M. F. Beg, M. I. Miller, A. Trouvé, and L. Younes, "Computing Large Deformation Metric Mappings via Geodesic Flows of Diffeomorphisms," *International Journal of Computer Vision*, vol. 61, no. 2, pp. 139–157, 2005, doi: 10.1023/B:VISI.0000043755.93987.aa.

[5] M. I. Miller, A. Trouvé, and L. Younes, "Hamiltonian Systems and Optimal Control in Computational Anatomy: 100 Years Since D'Arcy Thompson," *Annual Review of Biomedical Engineering*, vol. 17, pp. 447–509, 2015, doi: 10.1146/annurev-bioeng-071114-040601.

[6] S. C. Joshi and M. I. Miller, "Landmark matching via large deformation diffeomorphisms," *IEEE Transactions on Image Processing*, vol. 9, no. 8, pp. 1357–1370, 2000, doi: 10.1109/83.855431.